

ROS2 Day2 AI 사용 보고서

학번: 2026406076

이름: 정창규

제출일: 2026-09-10

목차

1. AI 사용 목적
2. Task01에서의 AI 활용
3. Task02에서의 AI 활용
4. Task03에서의 AI 활용
5. AI 사용 과정에서 느낀 점

1. AI 사용 목적

이번 ROS2 Day2 과제를 진행하면서 AI는 과제의 정답을 바로 얻기 위한 용도보다는 수업 자료에서 이해되지 않는 부분을 확인하고, 실습 과정에서 발생한 문제의 원인을 찾기 위한 용도로 사용하였다. Custom Message, Qt GUI, Publisher, Subscriber와 같은 내용을 다루면서 이전보다 구조가 복잡해졌기 때문에 수업 자료를 먼저 확인한 뒤 이해되지 않는 내용을 AI에게 질문하였다.

또한 Custom Message를 작성하거나 Qt Designer에서 만든 UI를 C++ 코드와 연결하는 과정에서 원하는 결과가 나오지 않을 때 AI를 활용하였다. AI가 제시한 내용을 바로 과제 결과로 사용하는 것보다 실제로 빌드하고 실행해본 뒤 결과가 맞는지 확인하고, 오류가 발생하면 필요한 부분을 다시 질문하거나 수정하는 방식으로 진행하였다.

2. Task01에서의 AI 활용

Task01을 진행하면서 수업 자료에서 Custom Message를 생성하는 방법을 먼저 확인하였다. 하지만 작성한 메시지를 Publisher와 Subscriber에서 실제로 사용하고, 입력받은 여러 값을 vector 형태로 전달하는 방법이 잘 이해되지 않아 AI에게 질문하였다.

AI의 설명을 통해 a에는 vector의 원소 개수를 저장하고, b에는 실제 데이터를 저장하는 방식으로 메시지를 구성할 수 있다는 것을 이해하였다. 또한 ros2 interface show 명령어를 이용하여 작성한 Custom Message가 정상적으로 생성되었는지 확인하는 방법도 알 수 있었다.

이후 Publisher와 Subscriber를 직접 실행하여 입력한 vector의 개수와 값이 정상적으로 전달되는지 확인하면서 Custom Message를 Topic 통신에서 사용하는 과정을 이해할 수 있었다.

Custom Message는 어떻게 구성하고, 입력받은 여러 값을 vector 형태로 Publisher에서 보내서 Subscriber에서 출력하려면 어떻게 해야 돼?

Custom Message는 먼저 .msg 파일에 전달할 데이터 형식을 정의하면 돼. 예를 들어 정수 하나와 여러 개의 정수를 함께 보내려면 이렇게 구성할 수 있어.

```
</> 일반 텍스트
int64 a
int32[] b
```

여기서 a에는 vector의 원소 개수를 저장하고, b에는 실제 vector 데이터를 넣으면 돼.

Publisher에서는 입력받은 값들을 `std::vector<int32_t>`에 저장한 뒤 메시지에 넣어서 publish하면 돼.

```
</> C++
std::vector<int32_t> data = {10, 20, 30, 40, 50};

msg.a = data.size();
msg.b = data;

publisher->publish(msg);
```

그러면 a에는 5, b에는 10, 20, 30, 40, 50이 들어가.

Subscriber에서는 같은 Custom Message 타입의 Topic을 구독하고 callback 함수에서 값을 읽으면 돼.

```
</> C++
void callback(const CustomMsg::SharedPtr msg)
{
    std::cout << "count: " << msg->a << std::endl;
    for (auto value : msg->b)
```

3. Task02에서의 AI 활용

Task02를 진행하면서 수업 자료에 나온 Qt GUI와 turtlesim 제어 방법을 먼저 확인하였다. 하지만 Qt Designer에서 만든 .ui 파일을 C++ 코드와 어떻게 연결하고, GUI의 버튼 입력을 실제 ROS2 동작과 어떻게 연결해야 하는지 이해가 부족하여 AI에게 질문하였다.

AI의 설명을 통해 QPushButton의 클릭 이벤트를 함수와 연결하는 방법과, W, A, S, D 버튼 및 키보드 입력을 이용하여 거북이를 이동시키는 구조를 이해할 수 있었다. 또한 linear.x, angular.z 값을 GUI에 표시하고, Triangle, Square, Circle 모드에 따라 선의 색상과 굵기를 변경하는 방법도 확인하였다.

이후 Qt Designer에서 직접 UI를 구성하고 코드를 빌드하여 버튼과 키보드 입력이 정상적으로 동작하는지 확인하였다. 이를 통해 Qt GUI에서 발생한 입력이 ROS2의 turtlesim 제어로 연결되는 과정을 이해하는 데 도움이 되었다.

Qt GUI의 WASD 버튼과 키보드 입력을 이용해서 turtlesim을 조종하고 linear.x와 angular.z 값까지 GUI에 표시하려면 어떻게 구현해야 돼?

Qt GUI에서 버튼 입력과 키보드 입력을 같은 함수로 연결하면 돼. 예를 들어 W는 전진, S는 후진, A/D는 좌우 회전 함수가 실행되도록 구성할 수 있어.

버튼은 `connect()` 를 이용해서 함수와 연결하면 돼.

```
</> C++  
  
connect(ui->wButton, &QPushButton::clicked,  
        this, [this]() {  
            moveForward();  
        });
```

키보드 입력은 `keyPressEvent()` 에서 처리할 수 있어.

```
</> C++  
  
void MainWindow::keyPressEvent(QKeyEvent *event)  
{  
    switch (event->key())  
    {  
        case Qt::Key_W:  
            moveForward();  
            break;  
  
        case Qt::Key_S:  
            moveBackward();  
            break;  
  
        case Qt::Key_A:  
            turnLeft();  
            break;  
  
        case Qt::Key_D:  
            turnRight();  
            break;  
    }  
}
```

4. Task03에서의 AI 활용

Task03를 진행하면서 수업 자료에 나온 Qt 기반 Talker와 Listener 통신 과정을 먼저 확인하였다. 하지만 Qt에서 입력한 문자열을 버튼을 눌러 Publish하고, Listener에서 수신한 데이터를 GUI의 Label에 표시하는 방법이 잘 이해되지 않아 AI에게 질문하였다.

AI의 설명을 통해 QLineEdit에 입력된 문자열을 가져와 /day2/chat Topic으로 발행하고, Listener의 callback 함수에서 수신한 문자열을 QLabel에 표시하는 구조를 이해하였다. 또한 Qt Designer에서 만든 .ui 파일을 C++ 코드와 연결하는 방법도 확인하였다.

이후 Talker와 Listener의 UI를 각각 직접 구성하고, Publish 버튼과 문자열 전송 기능을 연결하였다. Listener에서는 처음 Waiting...으로 표시되던 Label이 메시지를 수신하면 Talker에서 입력한 문자열로 변경되도록 구현하였다.

Task03에서는 AI를 통해 Qt GUI와 ROS2의 Topic 통신을 어떻게 연결하는지 이해할 수 있었고, 실제로 Talker와 Listener를 실행하여 문자열이 정상적으로 전달되고 Label 값이 변경되는지 확인하면서 코드를 수정하였다.

Qt에서 입력한 문자열을 Talker가 Publish하고 Listener가 Subscribe한 뒤 수신한 문자열로 QLabel의 값을 변경하려면 어떻게 구현해야 돼?

Qt에서는 Talker 쪽에서 QLineEdit에 입력된 문자열을 가져와 ROS2 Topic으로 Publish하고, Listener 쪽에서는 같은 Topic을 Subscribe한 뒤 callback 함수에서 QLabel의 내용을 바꾸면 돼.

Talker에서는 먼저 입력창의 문자열을 가져와 메시지에 넣어.

```
</> C++  
  
std_msgs::msg::String msg;  
  
msg.data =  
    ui->messageEdit->text().toString();  
  
publisher_->publish(msg);
```

이 코드를 Publish 버튼과 연결하면 버튼을 눌렀을 때 문자열이 전송돼.

```
</> C++  
  
connect(  
    ui->publishButton,  
    &QPushButton::clicked,  
    this,  
    [this]()  
    {  
        publishMessage();  
    });
```

Listener에서는 같은 Topic을 Subscribe하고, 메시지가 들어오면 callback 함수가 실행되도록 하면 돼.

```
</> C++  
  
subscription_ =  
    create_subscription<std_msgs::msg::String>(↓  
        "/day2/chat",  
        10,
```

5. AI 사용 과정에서 느낀 점

이번 ROS2 Day2 과제를 진행하면서 AI는 처음부터 답을 얻기 위한 용도보다는, 수업 자료를 보면서 이해되지 않는 부분을 질문하거나 실습 중 발생한 오류의 원인을 확인하는 데 주로 사용하였다. 특히 Custom Message를 이용한 vector 통신, Qt GUI와 ROS2의 연결, Talker와 Listener 사이의 Topic 통신처럼 처음에는 구조가 복잡하게 느껴졌던 부분을 하나씩 질문하면서 내용을 정리할 수 있었다.

AI의 설명을 통해 구현 방법을 확인한 뒤에는 그대로 사용하는 것이 아니라 실제로 코드를 빌드하고 실행하여 정상적으로 동작하는지 확인하였다. Qt 버튼을 눌렀을 때 turtlesim이 제대로 움직이는지, Talker에서 보낸 문자열이 Listener의 Label에 표시되는지 직접 확인하였고, 예상과 다른 결과가 나오면 코드나 설정을 다시 확인하면서 수정하였다. 이 과정을 통해 여러 기능이 서로 어떻게 연결되어 동작하는지 이해하는 것이 중요하다고 느꼈다.

또한 AI가 제시한 방법이 현재 과제의 환경이나 요구사항에 항상 정확히 맞는 것은 아니기 때문에 수업 자료와 실제 실행 결과를 함께 확인하는 과정이 필요하다는 점도 알게 되었다. 앞으로도 AI를 사용할 때에는 결과만 바로 사용하는 것보다 먼저 수업 자료를 확인하고, 이해되지 않는 부분이나 오류가 발생한 부분을 질문한 뒤 직접 실행하여 확인하는 방식으로 활용할 생각이다.